

**LAPORAN PRATIKUM STRUKTUR DATA
LINKED LIST DENGAN BAHASA JAVA**



Dosen Pengampu:
Wahyudi Dr.. S.T. M.T.

Disusun Oleh:
Kevin Maulana Sempri
2511531019

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2025**

Kata Pengantar

Puji syukur penulis ucapkan kepada Allah SWT karena penulis dapat menyelesaikan laporan praktikum ini dengan judul "Linked List Dengan Bahasa Java" dengan tepat waktu. Laporan ini dibuat sebagai salah satu untuk memenuhi nilai praktikum pada mata Struktur Data. Laporan ini berfokus pada studi kasus Linked List dengan menggunakan Bahasa Java. Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna, oleh karena itu penulis mengucapkan banyak terima kasih kepada Bapak / Ibu dosen dan para asisten, serta rekan-rekan mahasiswa yang telah banyak membantu penulis, dalam memberikan arahan & ilmu. Untuk perbaikan & kemajuan di masa yang akan datang, penulis mengharapkan kritik & saran yang membangun. Semoga laporan ini bisa memberikan manfaat & wawasan bagi pembaca, khususnya pada studi kasus Linked List menggunakan Bahasa Java. Akhir kata, penulis ucapkan terima kasih kepada semuanya.

DAFTAR PUSTAKA

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum.....	1
BAB II HASIL DAN PEMBAHASAN.....	2
2.1 Waktu dan Tempat.....	2
2.2 Cara Kerja.....	2
2.3 Hasil.....	3
2.4 Pembahasan	7
DAFTAR PUSTAKA.....	

Bab 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut adanya sistem penyimpanan data yang terstruktur dan mudah diakses. Struktur Data menjadi salah satu komponen penting dalam pengelolaan informasi, baik di bidang pendidikan, bisnis, maupun pemerintahan. Eclipse IDE merupakan salah satu perangkat lunak yang menyediakan fasilitas untuk merancang, mengelola, dan memanipulasi kode Bahasa pemrograman Java yang relatif mudah dipahami. Melalui praktikum ini, diharapkan mampu memahami Struktur Data Linked List serta mengimplementasikan studi kasus dengan menggunakan Bahasa Java.

1.2 Tujuan praktikum

1. Memahami konsep dasar struktur data dengan tipe data abstrak Linked List.
2. Mempelajari cara membuat struktur data Linked List dengan menggunakan Bahasa Java.
3. Mengimplementasikan struktur data sesuai kebutuhan studi kasus.
4. Melatih keterampilan dalam mengelola data secara sistematis dan efisien.

1.3 Manfaat praktikum

1. Memberikan pengalaman langsung dalam merancang dan mengelola struktur data.
2. Menambah wawasan tentang penggunaan struktur data dengan menggunakan Bahasa Java.
3. Mempersiapkan mahasiswa dengan keterampilan praktis yang dapat diterapkan dalam dunia kerja.
4. Membantu memahami pentingnya integritas dan konsistensi data dalam sistem informasi.

Bab 2 Hasil dan Pembahasan

Pada bab ini saya akan menampilkan hasil praktikum yang telah dilaksanakan beserta pembahasannya

2.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada hari Senin tanggal 5 Mei 2026 Dilaksanakan di laboratorium Jurusan Informatika, Fakultas Teknologi Infomarsi, Universitas Andalas.

2.2 Cara Kerja

Praktikan akan membuat sebuah program yang berupa Bahasa Pemrograman Java. Perintahnya adalah membuat kode untuk Abstract Data Types (Tipe data abstrak) dan kode untuk Driver (memanggil ADT tersebut)

2.3 Hasil

Kode Java NodeDLL:

```
package pekan6_2511531019;
public class NodeDLL_2511531019 {
    //mendefinisikan kelas Node
    int data_1019; // data
    NodeDLL_2511531019 next_1019; // Pointer ke next node
    NodeDLL_2511531019 prev_1019; // Pointer ke previous node

    //konstruktor
    public NodeDLL_2511531019(int data_1019) {
        this.data_1019 = data_1019;
        this.next_1019 = null;
        this.prev_1019 = null;
    }
}
```

Kode Java InsertDLL:

```
package pekan6_2511531019;
public class InsertDLL_2511531019 {
    static NodeDLL_2511531019 insertBegin_2511531019(NodeDLL_2511531019
head_1019, int data_1019) {
        NodeDLL_2511531019 new_node_1019 = new
NodeDLL_2511531019(data_1019);
        new_node_1019.next_1019 = head_1019;
        if (head_1019 != null) {
            head_1019.prev_1019 = new_node_1019;
        }
        return new_node_1019;
    }
    public static NodeDLL_2511531019
insertEnd_2511531019(NodeDLL_2511531019 head_1019, int newData_1019) {
        NodeDLL_2511531019 newNode_1019 = new
NodeDLL_2511531019(newData_1019);
        if (head_1019 == null) {
            head_1019 = newNode_1019;
        } else {
            NodeDLL_2511531019 curr_1019 = head_1019;
            while (curr_1019.next_1019 != null) {
                curr_1019 = curr_1019.next_1019;
            }
            curr_1019.next_1019 = newNode_1019;
            newNode_1019.prev_1019 = curr_1019;
        }
        return head_1019;
    }
    public static NodeDLL_2511531019
insertAtPosition_2511531019(NodeDLL_2511531019 head_1019, int pos_1019, int
new_data_1019) {
        NodeDLL_2511531019 new_node_1019 = new
NodeDLL_2511531019(new_data_1019);
        if (pos_1019 == 1) {
            new_node_1019.next_1019 = head_1019;
            if (head_1019 != null) {
                head_1019.prev_1019 = new_node_1019;
            }
            head_1019 = new_node_1019;
            return head_1019;
        }
        NodeDLL_2511531019 curr_1019 = head_1019;
        for (int i_1019 = 1; i_1019 < pos_1019 - 1 && curr_1019 != null;
++i_1019) {
            curr_1019 = curr_1019.next_1019;
        }
        if (curr_1019 == null) {
            System.out.println("Posisi tidak ada");
            return head_1019;
        }
        new_node_1019.prev_1019 = curr_1019;
        new_node_1019.next_1019 = curr_1019.next_1019;
        curr_1019.next_1019 = new_node_1019;
        if (new_node_1019.next_1019 != null) {
            new_node_1019.next_1019.prev_1019 = new_node_1019;
        }
    }
}
```

```

    }
    return head_1019;
}
public static void printList_2511531019(NodeDLL_2511531019 head_1019) {
    NodeDLL_2511531019 curr_1019 = head_1019;
    while (curr_1019 != null) {
        System.out.print(curr_1019.data_1019 + " <---> ");
        curr_1019 = curr_1019.next_1019;
    }
    System.out.println();
}
public static void main(String[] args) {
    NodeDLL_2511531019 head_1019 = new NodeDLL_2511531019(2);
    head_1019.next_1019 = new NodeDLL_2511531019(3);
    head_1019.next_1019.prev_1019 = head_1019;
    head_1019.next_1019.next_1019 = new NodeDLL_2511531019(5);
    head_1019.next_1019.next_1019.prev_1019 = head_1019.next_1019;

    System.out.print("DLL Awal: ");
    printList_2511531019(head_1019);

    System.out.print("Simpul 1 ditambah di awal: ");
    head_1019 = insertBegin_2511531019(head_1019, 1);
    printList_2511531019(head_1019);

    System.out.print("Simpul 6 ditambah di akhir: ");
    int data1_1019 = 6;
    head_1019 = insertEnd_2511531019(head_1019, data1_1019);
    printList_2511531019(head_1019);

    System.out.print("tambah node 4 di posisi 4: ");
    int data2_1019 = 4;
    int pos_1019 = 4;
    head_1019 = insertAtPosition_2511531019(head_1019, pos_1019,
data2_1019);
    printList_2511531019(head_1019);
}
}

```

Kode Java PenelusuranDLL:

```

package pekan6_2511531019;
public class PenelusuranDLL_2511531019 {
    static void forwardTraversal_2511531019(NodeDLL_2511531019 head_1019)
    {
        NodeDLL_2511531019 curr_1019 = head_1019;
        while (curr_1019 != null) {
            System.out.print(curr_1019.data_1019 + " <---> ");
            curr_1019 = curr_1019.next_1019;
        }
        System.out.println();
    }
    static void backwardTraversal_2511531019(NodeDLL_2511531019
tail_1019) {
        NodeDLL_2511531019 curr_1019 = tail_1019;
        while (curr_1019 != null) {

```

```

        System.out.print(curr_1019.data_1019 + " <---> ");
        curr_1019 = curr_1019.prev_1019;
    }
    System.out.println();
}
public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511531019 head_1019 = new NodeDLL_2511531019(1);
    NodeDLL_2511531019 second_1019 = new NodeDLL_2511531019(2);
    NodeDLL_2511531019 third_1019 = new NodeDLL_2511531019(3);
    head_1019.next_1019 = second_1019;
    second_1019.prev_1019 = head_1019;
    second_1019.next_1019 = third_1019;
    third_1019.prev_1019 = second_1019;
    System.out.println("Penelusuran Maju:");
    forwardTraversal_2511531019(head_1019);
    System.out.println("Penelusuran Mundur:");
    backwardTraversal_2511531019(third_1019);
}
}

```

Kode Java HapusDLL:

```

package pekan6_2511531019;
public class HapusDLL_2511531019 {
    public static NodeDLL_2511531019
delHead_2511531019(NodeDLL_2511531019 head_1019) {
        if (head_1019 == null) {
            return null;
        }
        NodeDLL_2511531019 temp_1019 = head_1019;
        head_1019 = head_1019.next_1019;
        if (head_1019 != null) {
            head_1019.prev_1019 = null;
        }
        return head_1019;
    }

    public static NodeDLL_2511531019
delLast_2511531019(NodeDLL_2511531019 head_1019) {
        if (head_1019 == null) {
            return null;
        }
        if (head_1019.next_1019 == null) {
            return null;
        }
        NodeDLL_2511531019 curr_1019 = head_1019;
        while (curr_1019.next_1019 != null) {
            curr_1019 = curr_1019.next_1019;
        }
        if (curr_1019.prev_1019 != null) {
            curr_1019.prev_1019.next_1019 = null;
        }
        return head_1019;
    }
}

```



```

        public static NodeDLL_2511531019 delPos_2511531019(NodeDLL_2511531019
head_1019, int pos_1019) {
            if (head_1019 == null) {
                return head_1019;
            }
            NodeDLL_2511531019 curr_1019 = head_1019;
            for (int i_1019 = 1; curr_1019 != null && i_1019 < pos_1019;
++i_1019) {
                curr_1019 = curr_1019.next_1019;
            }
            if (curr_1019 == null) {
                return head_1019;
            }
            if (curr_1019.prev_1019 != null) {
                curr_1019.prev_1019.next_1019 = curr_1019.next_1019;
            }
            if (curr_1019.next_1019 != null) {
                curr_1019.next_1019.prev_1019 = curr_1019.prev_1019;
            }
            if (head_1019 == curr_1019) {
                head_1019 = curr_1019.next_1019;
            }
            return head_1019;
        }
        public static void printList_2511531019(NodeDLL_2511531019 head_1019)
{
            NodeDLL_2511531019 curr_1019 = head_1019;
            while (curr_1019 != null) {
                System.out.print(curr_1019.data_1019 + " <---> ");
                curr_1019 = curr_1019.next_1019;
            }
            System.out.println();
        }
        public static void main(String[] args) {
            // TODO Auto-generated method stub
            NodeDLL_2511531019 head_1019 = new NodeDLL_2511531019(1);
            head_1019.next_1019 = new NodeDLL_2511531019(2);
            head_1019.next_1019.prev_1019 = head_1019;
            head_1019.next_1019.next_1019 = new NodeDLL_2511531019(3);
            head_1019.next_1019.next_1019.prev_1019 = head_1019.next_1019;
            head_1019.next_1019.next_1019.next_1019 = new
NodeDLL_2511531019(4);
            head_1019.next_1019.next_1019.next_1019.prev_1019 =
head_1019.next_1019.next_1019;
            head_1019.next_1019.next_1019.next_1019.next_1019 = new
NodeDLL_2511531019(5);
            head_1019.next_1019.next_1019.next_1019.next_1019.prev_1019 =
head_1019.next_1019.next_1019.next_1019;
            System.out.print("DLL Awal: ");
            printList_2511531019(head_1019);
            System.out.print("Setelah head dihapus: ");
            head_1019 = delHead_2511531019(head_1019);
            printList_2511531019(head_1019);
            System.out.print("Setelah node terakhir dihapus: ");
            head_1019 = delLast_2511531019(head_1019);
            printList_2511531019(head_1019);
            System.out.print("menghapus node ke 2: ");

```

```

        head_1019 = delPos_2511531019(head_1019, 2);
        printList_2511531019(head_1019);
    }
}

```

Output Kode Java InsertDLL

```

DLL Awal: 2 <---> 3 <---> 5 <--->
Simpul 1 ditambah di awal: 1 <---> 2 <---> 3 <---> 5 <--->
Simpul 6 ditambah di akhir: 1 <---> 2 <---> 3 <---> 5 <---> 6 <--->
tambah node 4 di posisi 4: 1 <---> 2 <---> 3 <---> 4 <---> 5 <---> 6 <--->

```

Output Kode Java PenelusuranDLL

```

Penelusuran Maju:
1 <---> 2 <---> 3 <--->
Penelusuran Mundur:
3 <---> 2 <---> 1 <--->

```

Output Kode Java HapusDLL

```

DLL Awal: 1 <---> 2 <---> 3 <---> 4 <---> 5 <--->
Setelah head dihapus: 2 <---> 3 <---> 4 <---> 5 <--->
Setelah node terakhir dihapus: 2 <---> 3 <---> 4 <--->
menghapus node ke 2: 2 <---> 4 <--->

```

2.4 Pembahasan

Praktikum ini menunjukkan bahwa Bahasa Java memudahkan proses membuat struktur data yang terdiri dari selector, mutator, dan lainnya. Dengan demikian, praktikum ini berhasil memberikan gambaran nyata tentang pentingnya Tipe Data Abstrak untuk struktur data dan driver sebagai eksekutor Tipe Data Abstrak untuk mendukung sistem informasi.

Bab 3 Kesimpulan

Praktikum ini berhasil menghasilkan struktur dalam data dengan menggunakan bahasa Java. Di dalam data memiliki struktur yang dapat menjalankan sebuah kasus kecil maupun besar untuk mendukung system informasi.